WEEK-09

Implement All Pair Shortest paths problem using Floyd's algorithm.

Pseudocode:

```
#define INF 99999

void floydWarshall(int dist[][V], int V)
{
   int i, j, k;
   for (k = 0; k < V; k++)
   {
      for (i = 0; i < V; i++)
      {
         for (j = 0; j < V; j++)
         {
            if (dist[i][k] + dist[k][j] < dist[i][j])
            {
               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }

   printf("Shortest distance matrix:\n");

   for (i = 0; i < V; i++)
   {
      for (j = 0; j < V; j++)
      {
         if (dist[i][j] == INF)
            printf("INF ");
         else
            printf("%d ", dist[i][j]);
      }
      printf("\n");
   }
}
```

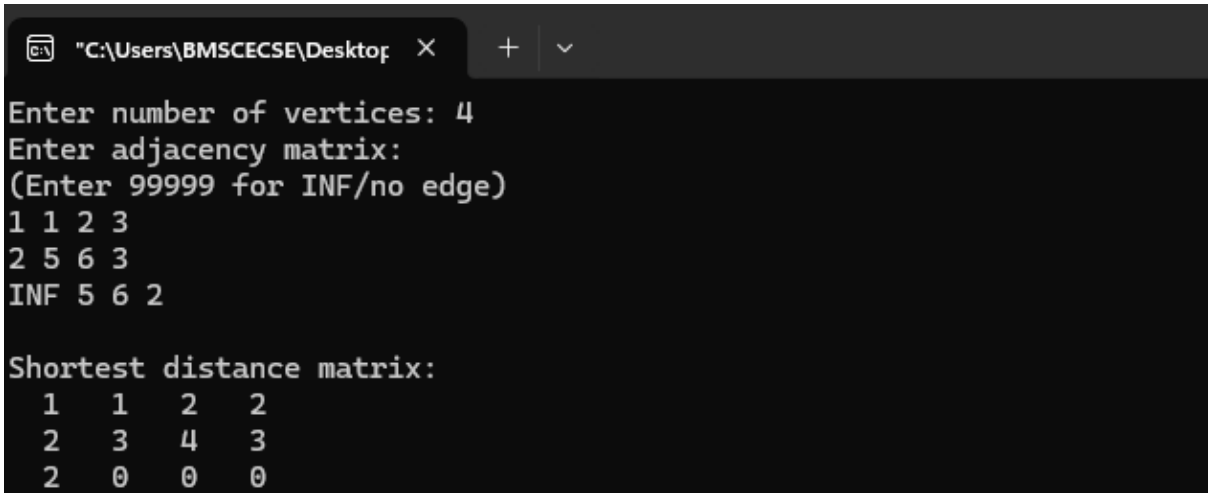
Program:

```
#include <stdio.h>

#define MAX 10
#define INF 99999

void floydWarshall(int dist[MAX][MAX], int V)
```

```
{
    int i, j, k;

    for (k = 0; k < V; k++)
    {
        for (i = 0; i < V; i++)
        {
            for (j = 0; j < V; j++)
            {
                if (dist[i][k] + dist[k][j] < dist[i][j])
                {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    printf("\nShortest distance matrix:\n");

    for (i = 0; i < V; i++)
    {
        for (j = 0; j < V; j++)
        {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%3d ", dist[i][j]);
        }
        printf("\n");
    }
}

int main()
{
    int V, i, j;
    int graph[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    printf("(Enter %d for INF/no edge)\n", INF);

    for (i = 0; i < V; i++)
    {
        for (j = 0; j < V; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
```

```
    floydWarshall(graph, V);

    return 0;
}
```

Output:



Leet Code exercise on Floyd's algorithm.

Find the City With the Smallest Number of Neighbors at a Threshold Distance

```
#define INF 1000000

int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int
distanceThreshold) {
    int dist[101][101];
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            dist[i][j] = (i == j) ? 0 : INF;
        }
    }
    for(int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];

        dist[u][v] = w;
        dist[v][u] = w;
    }
    for(int k = 0; k < n; k++)
        for(int i = 0; i < n; i++)
            for(int j = 0; j < n; j++)
                if(dist[i][k] + dist[k][j] < dist[i][j])
```

```
                    dist[i][j] = dist[i][k] + dist[k][j];

        int ans = -1, minCount = INF;
        for(int i = 0; i < n; i++) {

            int count = 0;

            for(int j = 0; j < n; j++)
                if(i != j && dist[i][j] <= distanceThreshold)
                    count++;

            if(count <= minCount) {
                minCount = count;
                ans = i;
            }
        }
        return ans;
    }
```